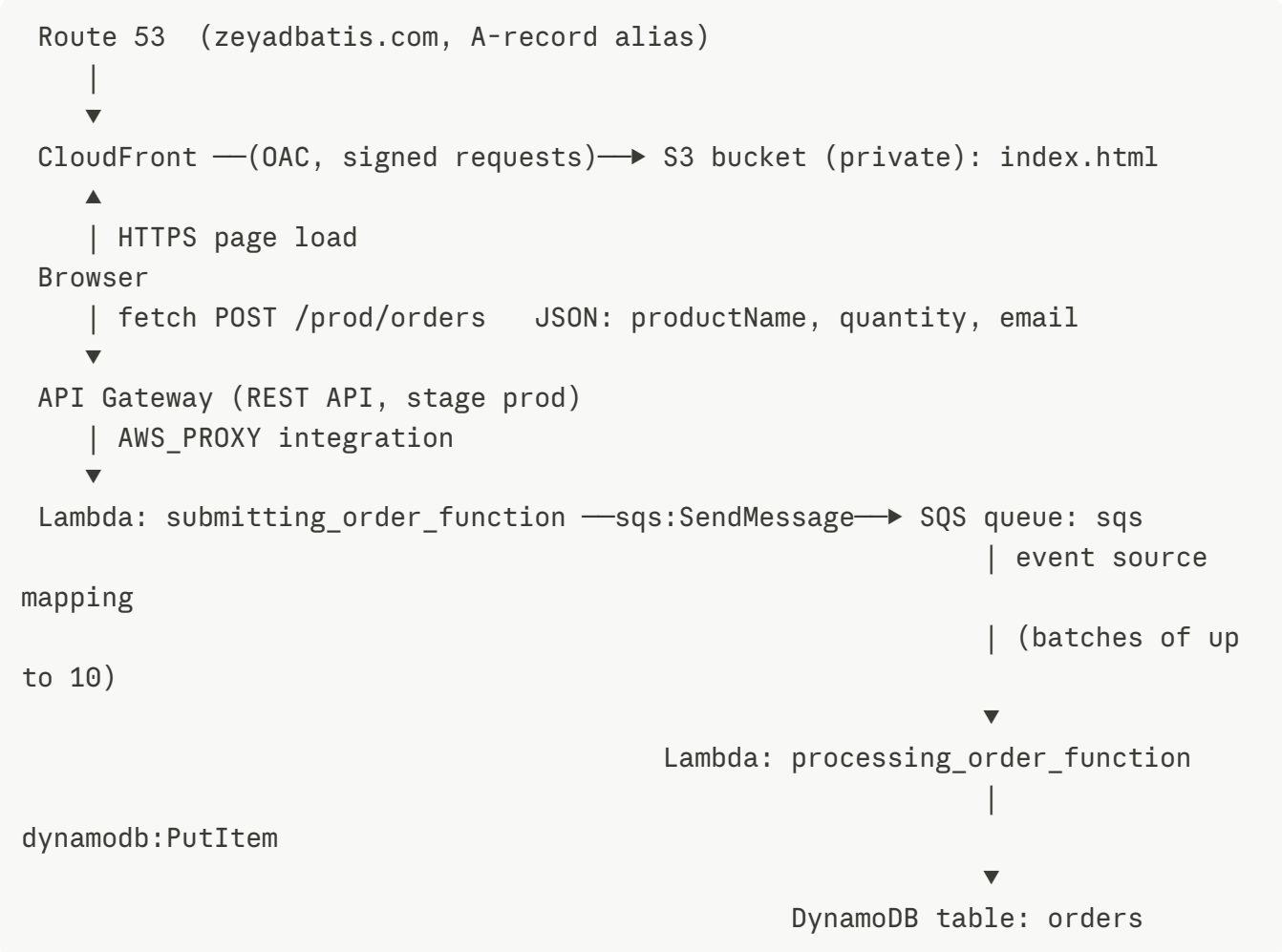
AWS Serverless Order Processing: Recipe and System Report

Terraform on AWS (ap-southeast-2) · Written 27 October 2026 from the code in your repository

1. The big picture

A customer fills in an order form on a static website. The form sends the order to an API. The API does the bare minimum (drops the order onto a queue and replies OK). A second function later picks orders off the queue and saves them to a database. The queue in the middle means a traffic spike or a slow database never makes the website feel broken.



There are four layers: **frontend hosting** (S3, CloudFront, Route 53), **API** (API Gateway), **compute and messaging** (two Lambdas, SQS) and **storage** (DynamoDB). Two IAM roles tie the permissions together, one per Lambda.

2. How it works: the life of one order

1. **Page load.** The browser asks for zeyadbatis.com. Route 53 has an alias record pointing at the CloudFront distribution.
2. **Serving the page.** CloudFront uses the ACM certificate (which must live in us-east-1) to serve HTTPS and redirects plain HTTP. It fetches index.html from the S3 bucket using Origin Access Control (OAC). The bucket blocks all public access and its policy only allows requests that come from this exact distribution. The file is then cached at the edge.
3. **Form validation.** JavaScript checks the product name is not empty, the quantity is a number above zero, and the email is not empty. It then calls fetch with POST and a JSON body.
4. **CORS preflight.** The page is on zeyadbatis.com but the API is on execute-api.amazonaws.com, so the browser first sends an OPTIONS request. API Gateway answers it itself with a MOCK integration that returns the CORS headers. No Lambda is invoked.
5. **The real request.** API Gateway matches the resource /orders and method POST. The AWS_PROXY integration hands the whole request to submitting_order_function. A Lambda permission allows only this API, this method and this path to invoke it.
6. **Submit Lambda.** It reads event body, converts it from JSON, sends it to SQS with send_message and returns 200 plus CORS headers. Its IAM role can only call sqs:SendMessage on this one queue (plus write logs).
7. **Queue.** SQS holds the message safely. An event source mapping polls the queue and calls processing_order_function with up to 10 messages in event Records, scaling to a maximum of 100 concurrent invocations.
8. **Processing Lambda.** For each record it builds an item: order_id is the SQS messageId, then productName, quantity, email and orderDate in UTC. It calls put_item on the orders table. If the function finishes without error, Lambda deletes the messages from the queue. If it throws, the messages reappear and are retried.
9. **Logs.** Both functions write to CloudWatch Logs through the AWSLambdaBasicExecutionRole managed policy.

Why this design is good. The API answers fast because it never touches the database. The queue absorbs bursts. A failure in processing does not lose orders. Using the SQS messageId as order_id also makes processing idempotent: if SQS delivers a message twice, the second put_item overwrites the same row instead of creating a duplicate.

3. Prerequisites: things Terraform does not create

Your code looks these up or relies on them, so they must exist before the first apply. This is the most common reason a rebuild fails.

What	Why it is needed	Where it is used
Terraform 1.10 or newer	use_lockfile (native S3 state locking) needs it	backend block in maint.tf
AWS CLI with working credentials	Terraform and your tests use them	everywhere
S3 bucket zeyad-terraform-state-files in ap-southeast-2	Stores remote state. Turn on versioning.	backend block
Domain zeyadbatis.com and a Route 53 hosted zone	Looked up with a data source	route53.tf
ACM certificate for the domain in us-east-1, status ISSUED	CloudFront only accepts certificates from us-east-1	frontend.tf data source

4. Project layout

Path	Purpose
terraform/maint.tf	Terraform and provider versions, S3 backend, two AWS providers (default region and us_east_1), local for the S3 origin id
terraform/variables.tf	aws_region, bucket_name, api_http_method, domain_name
terraform/terraform.tfvars	Your values (bucket_name)
terraform/dynamodb.tf	The orders table
terraform/sqs.tf	The queue and the event source mapping to the processing Lambda
terraform/iam_role.tf	Both Lambda roles and their policies
terraform/lambda.tf	Zips the Python files and creates both functions
terraform/api_gw.tf	REST API, /orders resource, POST and OPTIONS methods, deployment, stage, Lambda permission

Path	Purpose
terraform/frontend.tf	S3 bucket, public access block, OAC, bucket policy, index.html upload, CloudFront distribution
terraform/route53.tf	DNS alias record to CloudFront
terraform/output.tf	Prints API_URL and SQS_URL
lambda/	SubmitOrderFunction.py and ProcessOrderFunction.py (source code)
frontend/index.html	The order form

5. The recipe: build it in this order

Terraform works out the real creation order itself from references. This is the order to think and build in, so each step has what it depends on already in place.

5.1 Foundation

Goal: tell Terraform which providers to use and where to keep state.

Block	Key settings
terraform / required_providers	hashicorp/aws version ~> 6.0
terraform / backend s3	bucket, key aws-serverless-order-processing/terraform.tfstate, region, encrypt = true, use_lockfile = true
provider aws	region ap-southeast-2
provider aws, alias us_east_1	region us-east-1, used only to find the ACM certificate
variables	aws_region, bucket_name, api_http_method (POST), domain_name
outputs	API_URL (stage invoke_url plus /orders), SQS_URL

Watch out: the archive provider is used by the Lambda zips but is not declared in required_providers. Declare it.

5.2 DynamoDB (the data)

Resource	Key arguments
aws_dynamodb_table.orders_table	name orders, billing_mode PAY_PER_REQUEST, hash_key email, range_key order_id, both attributes type S

Design: partition key is the customer email and sort key is the order id, so one query returns all orders for a customer. PAY_PER_REQUEST means no capacity planning and no cost when idle.

Watch out: the comment says streams are enabled but the value is stream_enabled = false. Fix the comment or the value.

5.3 SQS (the buffer)

Resource	Key arguments
aws_sqs_queue.sqs	name sqs, all defaults (30 second visibility timeout, 4 day retention)

The event source mapping that connects it to the processing Lambda is created in step 5.6, because it needs the Lambda to exist first.

5.4 IAM (who may do what)

Build one role per Lambda. Each role has three parts: a trust policy saying Lambda may assume it, inline policies with only what the function needs, and the AWS managed logging policy.

Resource	Purpose
data.aws_iam_policy_document.submitting_role_policy	Trust policy: principal lambda.amazonaws.com, action sts:AssumeRole
aws_iam_role.submitting_role	Role for the submit function
aws_iam_role_policy.submiting_sqs	Allow sqs:SendMessage on the queue ARN only
aws_iam_role_policy_attachment.submiting_logs	AWSLambdaBasicExecutionRole (CloudWatch logs)

Resource	Purpose
data.aws_iam_policy_document.processing_role_policy	Same trust policy for the second role
aws_iam_role.processing_role	Role for the processing function
aws_iam_role_policy.processing_sqs	Allow ReceiveMessage, DeleteMessage, GetQueueAttributes on the queue ARN
aws_iam_role_policy.processing_dynomodb	Allow dynamodb:PutItem on the table ARN only
aws_iam_role_policy_attachment.processing_logs	AWSLambdaBasicExecutionRole

Pattern: every Resource is a Terraform reference (aws_sqs_queue.sqs.arn), never a typed ARN. This is least privilege done properly.

5.5 Lambda (the code)

Repeat this three-part pattern per function:

Resource	Key arguments
data.archive_file.submitting_order	type zip, source_file ../lambda/SubmitOrderFunction.py, output_path lambda/SubmitOrderFunction.zip
aws_lambda_function.submitting_order_function	filename (the zip), function_name, role (submitting_role ARN), handler SubmitOrderFunction.lambda_handler, source_code_hash (zip hash), runtime python3.12
data.archive_file.processing_order	same pattern for ProcessOrderFunction.py
aws_lambda_function.processing_order_function	role processing_role ARN, handler ProcessOrderFunction.lambda_handler

Key idea: handler is the Python file name, a dot, then the function name. source_code_hash is what makes Terraform redeploy when the Python changes.

5.6 Queue to processing Lambda

Resource	Key arguments
aws_lambda_event_source_mapping.sqs	event_source_arn (queue ARN), function_name (processing function ARN), batch_size 10, scaling_config maximum_concurrency 100

5.7 API Gateway (the front door)

This is a REST API (v1). It is made of many small pieces, and CORS needs four more resources.

Resource	What it does
aws_api_gateway_rest_api.lambda_api	The API container
aws_api_gateway_resource.resource	The path /orders under the root
aws_api_gateway_method.method	POST on /orders, authorization NONE
aws_api_gateway_integration.integration	type AWS_PROXY, integration_http_method POST, uri built from region and the submit function ARN
aws_lambda_permission.api_gateway	Lets apigateway.amazonaws.com invoke the submit function, source_arn limited to this API, any stage, POST, /orders
aws_api_gateway_method.options	OPTIONS on /orders for the browser preflight
aws_api_gateway_integration.options	type MOCK, returns status 200 without calling Lambda
aws_api_gateway_method_response.options	Declares the three Access-Control headers
aws_api_gateway_integration_response.options	Sets header values: origin https://zeyadbatis.com, methods OPTIONS,POST, headers Content-Type,Authorization
aws_api_gateway_deployment.deployment	A snapshot of the API. triggers hashes every method and integration so a change forces a redeploy. lifecycle create_before_destroy.

Resource	What it does
aws_api_gateway_stage.api_stage	Stage named prod pointing at the deployment. Its invoke_url is your API address.

Why the permission matters: without aws_lambda_permission the API returns 500 even though everything else looks right.

Two places need CORS: the OPTIONS mock answers the preflight, and the Lambda itself must also return the Access-Control-Allow-Origin header on the real POST response.

5.8 Frontend hosting

Resource	What it does
aws_s3_bucket.first_bucket	Bucket for the website (name from var.bucket_name)
aws_s3_bucket_public_access_block	All four block settings true, so the bucket is fully private
aws_cloudfront_origin_access_control.oac	s3 origin type, signing_behavior always, sigv4
data.aws_iam_policy_document.allow_cloudfront	Allow s3:GetObject to cloudfront.amazonaws.com, only when AWS:SourceArn equals the distribution ARN
aws_s3_bucket_policy.allow_cloudfront	Attaches that policy. depends_on the public access block.
aws_s3_object.index_html	Uploads frontend/index.html, content_type text/html, etag filemd5 so edits re-upload
data.aws_cloudfront_cache_policy.caching_optimized	Looks up the AWS managed cache policy by name
data.aws_acm_certificate.my_domain	Finds the issued certificate in us-east-1 (uses the aliased provider)

Resource	What it does
aws_cloudfront_distribution.s3_distribution	Origin is the bucket regional domain with OAC. aliases is the domain, default_root_object index.html, viewer_protocol_policy redirect-to-https, price_class PriceClass_100, sni-only certificate

5.9 DNS

Resource	What it does
data.aws_route53_zone.my_domain	Finds your hosted zone by name
aws_route53_record.cloudfront	A record (alias) from the domain to the CloudFront domain name and hosted zone id. Created with for_each over the distribution aliases.

5.10 Connect the frontend to the API

The form calls the API using a URL typed into index.html. That URL only exists after the API is created, so the first build is two passes: apply, copy the API_URL output into index.html, then apply again. Section 9 shows how to remove this manual step.

6. Dependency map: what references what

From	Uses	Why
IAM sqs policies	SQS queue ARN	Scope permissions to one queue
processing_dynomodb policy	DynamoDB table ARN	Scope PutItem to one table
Both Lambda functions	Their IAM role ARNs	Execution identity
Lambda functions	archive_file output	Code package and change detection
Event source mapping	Queue ARN and processing function ARN	Wires queue to compute

From	Uses	Why
API integration and Lambda permission	Submit function ARN	Route requests to the function
Deployment triggers	All method and integration ids	Redeploy on change
Stage	Deployment id	Publish the API
Bucket policy	CloudFront distribution ARN	Only this distribution can read
CloudFront distribution	Bucket domain, OAC id, ACM certificate ARN	Origin, security, HTTPS
Route 53 record	CloudFront domain and zone id, hosted zone id	DNS alias
Outputs	Stage invoke_url, queue URL	Information for you

Not references (hardcoded in code): the queue URL inside SubmitOrderFunction.py, the table name in ProcessOrderFunction.py and the API URL in index.html. These are the main things to fix (section 9).

7. Deploy, test, destroy

```
cd terraform
terraform init
terraform fmt -check
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
terraform output
```

Test the whole pipeline

```
curl -X POST "$(terraform output -raw API_URL)" -H 'Content-Type:
application/json' -d
'{"productName":"Keyboard","quantity":2,"email":"me@example.com"}'
aws dynamodb scan --table-name orders --region ap-southeast-2
aws logs tail /aws/lambda/submitting_order_function --follow --region ap-
southeast-2
aws logs tail /aws/lambda/processing_order_function --follow --region ap-
southeast-2
```

Troubleshooting guide

Symptom	Likely cause
Plan fails on the ACM or Route 53 data source	Certificate not issued in us-east-1, or no hosted zone
Browser shows a CORS error	Origin in OPTIONS response or Lambda headers does not match the page origin exactly
API returns 500 Internal Server Error	Missing aws_lambda_permission, or Lambda crashed
Order accepted but nothing in DynamoDB	Check processing logs. A missing field raises KeyError and the batch retries.
Page does not update after editing index.html	CloudFront cache. Create an invalidation: aws cloudfront create-invalidation --distribution-id ID --paths /index.html
Frontend still calls an old API	The hardcoded API_URL in index.html was not updated

Destroy: run terraform destroy. The state bucket, hosted zone and certificate are not managed by this project, so they remain. CloudFront deletion takes several minutes.

8. Resource count

About 35 objects: 2 archive data sources, 2 Lambdas, 2 roles, 3 inline policies, 2 policy attachments, 1 queue, 1 mapping, 1 table, 11 API Gateway resources, and around 9 for hosting and DNS, plus several data sources for lookups. The tables in section 5 list every one.

9. Review: what is strong, and what to fix next

Already done well

- Remote state in S3 with encryption and native locking. This is a step many beginners skip.
- One role per Lambda, with policies scoped to exact resource ARNs.
- Private bucket with OAC and a bucket policy locked to one distribution.
- Deployment triggers and create_before_destroy on API Gateway, source_code_hash on Lambdas, etag on the S3 object. These show you understand how Terraform detects change.
- Queue-based decoupling, and the idempotent messageId design.

- CORS handled in both places it is needed.

Fix next, in priority order

1. **Remove hardcoded values.** Pass the queue URL and table name to the Lambdas with an environment block (QUEUE_URL = aws_sqs_queue.sqs.url, TABLE_NAME = aws_dynamodb_table.orders_table.name) and read them with os.environ. Your AWS account id is currently inside the Python file. For the frontend, make index.html a template with a placeholder and use templatefile() when uploading it, so the API URL is injected automatically and the two-pass build disappears.
2. **Add a dead-letter queue.** Right now one bad message (for example a missing productName) fails the whole batch and retries until the 4 day retention ends. Add a second queue and a redrive_policy with maxReceiveCount of about 3-5. Also set function_response_types to ReportBatchItemFailures on the mapping and return batchItemFailures from the function, so one bad record does not retry the other nine.
3. **Set timeouts explicitly.** Lambda defaults to 3 seconds. Set timeout and memory_size on both functions and make the queue visibility_timeout_seconds comfortably larger than the processing function timeout (AWS suggests at least six times).
4. **Validate on the server.** The browser checks can be bypassed. In the submit Lambda check the JSON, required fields, quantity and email format. Return 400 for bad input and 500 for system failures, and do not return str(e) to the caller.
5. **Protect the API.** It is open to anyone who finds the URL, and CORS only affects browsers. Add throttling (method settings or a usage plan), and consider AWS WAF or authentication later.
6. **Use the variable consistently.** The method uses var.api_http_method but the Lambda permission source_arn hardcodes POST.
7. **Operations.** Create aws_cloudwatch_log_group resources with a retention period, add an alarm on dead-letter queue depth, and add default_tags to the provider.
8. **Housekeeping.** Rename maint.tf to main.tf, add required_version and the archive provider, remove the unused availability zones data source, add *.zip to .gitignore (the built Lambda zips are build output), give bucket_name a valid default (S3 names cannot contain underscores), add an AAAA record for IPv6, and write the README (architecture diagram, prerequisites, deploy steps).

10. What to learn next

You already have remote state and locking, least-privilege IAM and a multi-provider setup. The next skills build directly on this project.

1. **Variables, locals, environment injection and validation blocks.** Finish the clean-up in section 9 point 1. This is the highest value step.

2. **Reliability patterns.** Dead-letter queues, partial batch responses, alarms. Then try Step Functions or EventBridge to extend the order flow (for example send a confirmation email with SES after saving).
3. **Modules.** Turn the repeated pattern (role, policy, function, log group) into a reusable lambda module and call it twice. Then read how community modules are built.
4. **Environments.** Run dev and prod from the same code, with separate state keys and variable files.
5. **CI/CD.** GitHub Actions running fmt, validate and plan on pull requests and apply on merge, authenticating to AWS with OIDC instead of access keys.
6. **Quality and security scanning.** tflint, checkov or trivy and pre-commit hooks.
7. **State skills.** import blocks, moved blocks, state commands, drift detection, lifecycle rules. Also bootstrap the state bucket itself with Terraform in a small separate project.
8. **Testing.** The native terraform test command, then Terratest.
9. **Broaden.** Try an HTTP API (API Gateway v2), then OpenTofu and AWS CDK to compare IaC approaches.

11. Terraform patterns used in this project (cheat sheet)

Pattern	Where you used it
Remote backend with locking	maint.tf
Provider alias for another region	us_east_1 for ACM
Data source lookups	ACM certificate, Route 53 zone, CloudFront cache policy, IAM policy documents
Resource references for wiring	ARNs and ids everywhere instead of typed values
Variables and tfvars	bucket_name, domain_name
locals	s3_origin_id
archive_file plus source_code_hash	Lambda packaging
filemd5 etag	index.html upload
triggers and lifecycle	API Gateway deployment
depends_on	Bucket policy after public access block, CORS pieces before deployment
for_each	Route 53 record from distribution aliases
jsonencode	Inline IAM policies

Pattern	Where you used it
outputs	API_URL and SQS_URL